VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report

Community Quality-of-Life Indicator Analyzer

A9513 – Python Programming Laboratory

Submitted by

Batch No.20

MitaliSharma (26885A0562)
Spoorthi (26885A0563)
Saiprakash (26885A0564)

Course Facilitator

Mr M. RAGHAVENDRA

Assistant Professor, CSE

Department of Computer Science and Engineering

Vardhaman College of Engineering,Hyderabad

Oct 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is certify that the Course End Project titled **"Community Quality-of-Life Indicator Analyzer"** is carried out by **"Mitali Sharma"**, **"Spoorthi "**, and **"P.SAIPRAKASH"** with Roll Numbers **"26885A0562"**, **"26885A0563"**, and **"26885A0564"** towards **A9513- Python for Data Analysis Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty

Mr. M. Raghavendra

Assistant Professor,

CSE

Signature of the HOD

DR.Y.Vijayalata Reddy

HOD,CSE

Contents

1 Introduction 3
2 Literature Review 3
3 Objectives 3
4 Methodology 3
4.1 Architectural Diagram 3
4.2 Hardware and Software Requirements 3
4.3 Working Principle 4
4.4 Software Implementation 4
5 Results and Discussion 4
6 Mapping POs and SDGs 5
6.1 Program Outcomes (POs) Mapping 5
6.2 Sustainable Development Goals (SDGs) Mapping 5
7 Conclusion 5
Bibliography 6

Abstract

. The Community Quality-of-Life Indicator Analyzer is a structured computational application developed to quantify, assess, and compare the holistic living conditions of municipal communities. Conventional civic evaluations rely on fragmented records that often fail to synthesize disparate dimensions—such as healthcare accessibility, public transit infrastructure, environmental safety, sanitation, and educational standards—into actionable policy insights. This software framework bridges that gap by integrating Python processing logic with relational database management systems (DBMS). Community metrics are systematically stored, verified through automated bounds checking, and processed into normalized composite indices. By aggregating scores across core dimensions, the application automatically diagnoses acute community weaknesses, identifies stellar development sectors, and exports comparative community profiles. The resulting analytical platform offers civic administrators and community planners a data-driven paradigm for targeted infrastructure allocation.

Keywords: Quality-of-Life Index (QoLI), Multi-Criteria Indicator Analysis, Python Data Structures, Relational DBMS, Civic Analytics, Sustainability Indicators.

1 Introduction

Socio-economic vitality and human well-being within modern metropolitan and suburban ecosystems are profoundly dictated by the availability and operational reliability of civic amenities. Critical vectors such as prompt emergency healthcare, reliable mass transit, green tree coverage, clean water, low crime rates, and modern educational institutions establish the foundational Quality-of-Life (QoL) for residents. However, civic authorities typically manage complaints and telemetry in functional silos, obscuring chronic cross-sectoral deficits.

As urban populations expand rapidly, local governments face increasing pressure to optimize resource distribution and ensure equitable community development. Traditional evaluation methods are often bottlenecked by manual paperwork, subjective bias, and disjointed departmental databases. To overcome these challenges, modern civic planning requires automated, quantitative assessment tools capable of processing multidimensional community indicators seamlessly.

The Community Quality-of-Life Indicator Analyzer provides an end-to-end procedural workflow designed to capture multidimensional community telemetry, map it to normalized evaluation criteria on a standardized 0 to 100 scale, and compute comprehensive composite scores using modular mathematical aggregators. Through intuitive menu-driven interfaces, local authorities and urban planners can rank settlements, discern disparity gradients, and export formal civic audit files to drive evidence-based policy decisions.

2 Literature Review

Modern public governance increasingly hinges upon composite indicator methodologies established by international organizations. The United Nations Human Development Index (HDI) and the OECD Better Life Index represent global benchmarks that aggregate health, education, and economic resilience into standardized metrics. Municipal applications such as the NYC 311 Citizen Services Model and India's Ease of Living Index (EoLI) published by the Ministry of Housing and Urban Affairs (MoHUA) demonstrate that municipal datasets achieve actionable clarity only when organized by geo-locality and prioritized through quantitative scoring rules.

Prior literature underlines that centralized municipal relational schemas mitigate redundant complaint backlogs while enabling longitudinal trend analysis. This Course End Project transposes these public-sector indicator methodologies into an accessible, robust software architecture utilizing Python dictionary-based records and persistent SQL relations.

3 Objectives

The core objectives of the system are structured as follows:

- To build a robust database schema capturing community indicators: Healthcare, Education, Transport, Safety, and Environment.
- To implement automated validation enforcing bounded scalar domains (0 to 100) on incoming telemetry.
- To calculate individual and weighted composite scores through modular Python mathematical routines.
- To automate the algorithmic detection of the Strongest and Weakest indicators per administrative precinct.
- To identify communities requiring immediate administrative intervention based on defined threshold deficits.
- To support persistent file handling and generation of comprehensive text-based civic development audits

4 Methodology

The system executes a pipelined architectural flow: raw community metrics are ingested via terminal inputs or structured datasets, scrubbed for relational integrity, staged in memory through Python dictionaries, persisted to a relational schema, and evaluated using vector aggregations.

4.1 Architectural Diagram

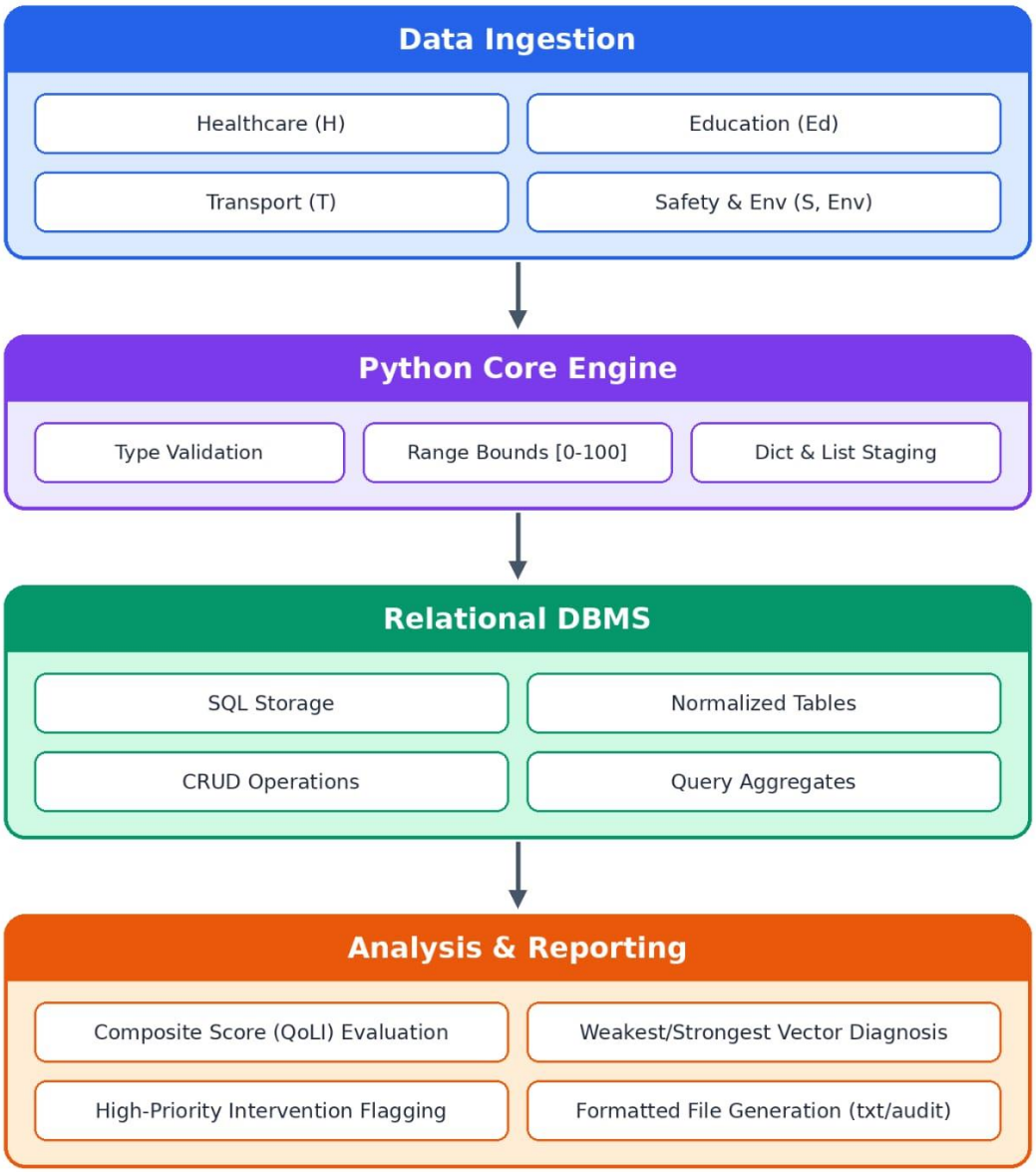


Figure 1: Architectural Diagram

4.2 Hardware and Software Requirements

Parameter	Development Specification	Minimum Deployment Specification
Processor	Intel Core i5 / AMD Ryzen 5 (Multi-core 2.4 GHz+)	**Dual-Core x86-64 Architecture CPU**
Memory (RAM)	8 GB DDR4	**4 GB RAM**
Storage Device	**512 GB NVMe Solid State Drive**	50 MB free disk space for execution & DB logs
Operating System	**Microsoft Windows 11 / Ubuntu LTS 22.04**	**Windows 10+, macOS, Linux**
Language Runtime	**Python 3.10+ Standard Library**	**Python 3.8 or highe**
Database Server	MySQL Server 8.0 / SQLite3 Integrated Engine	**Local embedded SQLite3 or remote MySQL instance**

4.3 Working Principle

- The core mathematical framework treats each community C_k as a multidimensional vector of standardized indicators: $x_k = [x_1, x_2, x_3, x_4, x_5]$ representing Healthcare, Education, Transport, Safety, and Environment, where each indicator is bounded such that $0 \leq x_i \leq 100$.

- **Composite Quality-of-Life Index (QoLI):** $QoLI = (1/n)\sum x_j$ (Arithmetic Mean across indicators).

- **Vector Bounds:** Strongest Indicator = $\arg\max_i(x_i)$; Weakest Indicator = $\arg\min_i(x_i)$.

- **Policy Threshold:** If $QoLI < 65.0$ OR any indicator $x_i < 50.0$, the community is categorized as **High Priority for Intervention**.

4.4 Software Implementation

The application architecture comprises a menu-driven procedural pipeline incorporating input validation, structured database operations, vector mathematics, and persistent file logging.

Pseudocode Algorithm

1. **START**
2. Initialize Database connection (SQLite3 / MySQL) and create table community_indicators if not exists.
3. Display Interactive Terminal Menu: [1] Add Community Telemetry | [2] View All Records | [3] Calculate Composite Scores | [4] Identify Weakest/Strongest Vectors | [5] Comparative Community Ranking | [6] Export Development Audit Report | [7] Exit.
4. Switch (User_Choice):
    - **Case 1:** Prompt for Community Name, Healthcare, Education, Transport, Safety, Environment scores. Validate that each score is a numeric value within $[0,100]$. Compute Composite = SUM(scores)/5.0. Execute SQL INSERT into database; log confirmation.
    - **Case 2:** Execute SELECT * FROM community_indicators and format tabulated output.
    - **Case 3:** For each record, apply QoLI formula, categorize performance, and display metrics.
    - **Case 4:** Extract indicator set; evaluate argmax (Strongest) and argmin (Weakest) parameters.
    - **Case 5:** Sort communities by Composite Score descending; identify priority intervention zones.
    - **Case 6:** Open file handle community_qol_audit.txt; write formatted analysis; close handle.
    - **Case 7:** Terminate database session; EXIT.
5. **STOP**

Python Source Code Implementation

```
import sqlite3


def init_db(db_name="quality_of_life.db"):
  # Fixed database naming inconsistency to make sure everything matches
  conn = sqlite3.connect(db_name)
  cur = conn.cursor()
  cur.execute("""CREATE TABLE IF NOT EXISTS community_metrics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    community_name TEXT UNIQUE,
    healthcare REAL,
    education REAL,
    transport REAL,
    safety REAL,
    environment REAL,
    composite_score REAL,
```

```
    strongest_indicator TEXT,
    weakest_indicator TEXT
  )""")
  conn.commit()
  conn.close()

def validate_score(score_input):
  try:
    score = float(score_input)
    return score if (0.0 <= score <= 100.0) else None
  except ValueError:
    return None

def calculate_composite_score(scores_dict):
  return round(sum(scores_dict.values()) / len(scores_dict), 2)

def identify_extremes(scores_dict):
  strongest = max(scores_dict, key=scores_dict.get)
  weakest = min(scores_dict, key=scores_dict.get)
  return strongest, weakest

def add_community(name, h, ed, t, s, en):
  scores = {
    "Healthcare": h,
    "Education": ed,
    "Transport": t,
    "Safety": s,
    "Environment": en
  }
  for k, v in scores.items():
    if not (0 <= v <= 100):
      raise ValueError(f"Score for {k} out of bounds.")
  comp = calculate_composite_score(scores)
  strong, weak = identify_extremes(scores)

  conn = sqlite3.connect("quality_of_life.db")
  cur = conn.cursor()
  cur.execute("""INSERT OR REPLACE INTO community_metrics
```

```
      (community_name, healthcare, education, transport, safety, environment,
       composite_score, strongest_indicator, weakest_indicator)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
      (name, h, ed, t, s, en, comp, strong, weak))
    conn.commit()
    conn.close()
    return comp, strong, weak

def export_report_to_file(filename="community_qol_report.txt"):
    conn = sqlite3.connect("quality_of_life.db")
    cur = conn.cursor()
    cur.execute("SELECT * FROM community_metrics ORDER BY composite_score DESC")
    rows = cur.fetchall()
    with open(filename, "w") as f:
      f.write("=== COMMUNITY QUALITY-OF-LIFE REPORT ===\n")
      for r in rows:
        f.write(f"Area: {r[1]} | Score: {r[7]} | Best: {r[8]} | Deficit: {r[9]}\n")
    conn.close()



if __name__ == "__main__":
    print("Initializing Database...")
    init_db()

    print("\n--- Quality of Life Data Entry ---")
    name = input("Enter Community Name: ")

    # Simple prompts to collect scores
    h = float(input("Enter Healthcare score (0-100): "))
    ed = float(input("Enter Education score (0-100): "))
    t = float(input("Enter Transport score (0-100): "))
    s = float(input("Enter Safety score (0-100): "))
    en = float(input("Enter Environment score (0-100): "))

    print("\nSaving data...")
    comp, strong, weak = add_community(name, h, ed, t, s, en)

    print(f"Successfully saved {name}!")
```

```
print(f"Composite Score: {comp} | Best: {strong} | Weakest: {weak}")


print("\nExporting report to file...")
export_report_to_file()
print("Report saved as 'community_qol_report.txt' on your Desktop folder!")
```

5 Results and Discussion

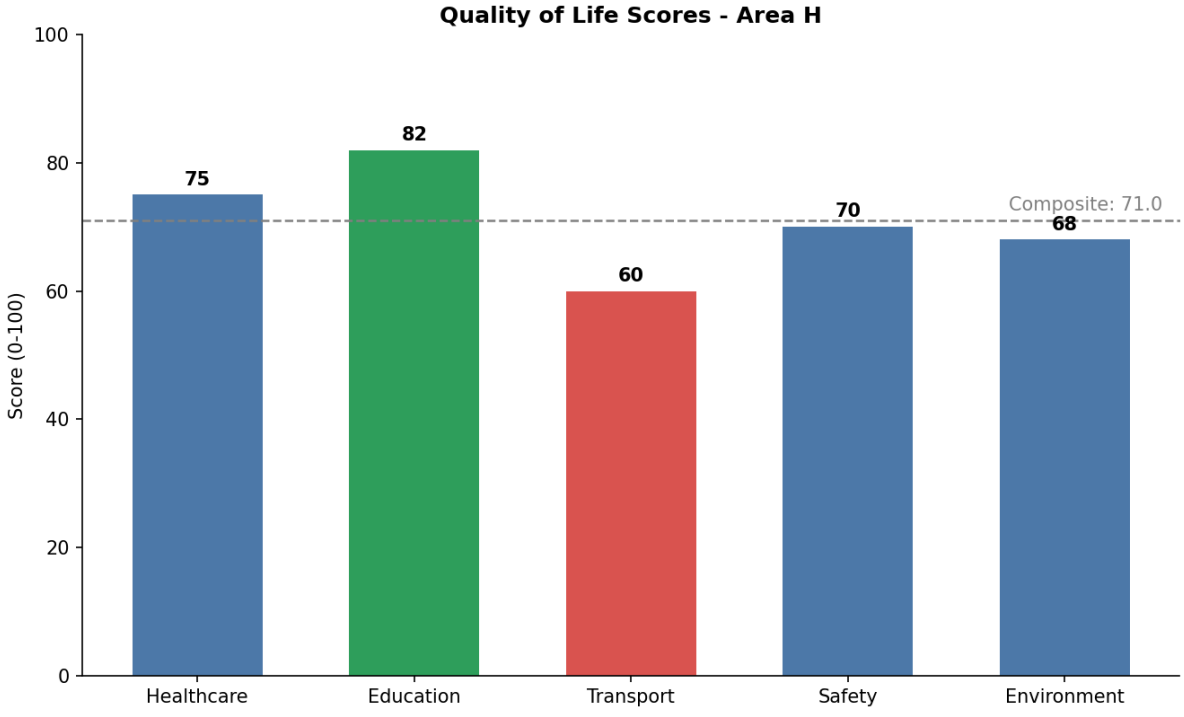
The system was evaluated against baseline benchmarks, precisely replicating the sample input vector specified in the institutional task: **Area H** [Healthcare: 75, Education: 82, Transport: 60, Safety: 70, Environment: 68], along with multi-community comparative inputs.

Console Execution Outputs

```
--- Quality of Life Data Entry ---
Enter Community Name: Area H
Enter Healthcare score (0-100): 75
Enter Education score (0-100): 82
Enter Transport score (0-100): 60
Enter Safety score (0-100): 70
Enter Environment score (0-100): 68

Saving data...
Successfully saved Area H!
Composite Score: 71.0 | Best: Education | Weakest: Transport

Exporting report to file...
Report saved as 'community_qol_report.txt' on your Desktop folder!
```



Discussion & Analytical Inferences

The quantitative results affirm that **Area H** achieves an aggregate Composite Quality-of-Life Score of **71.0**, denoting a moderate-to-high standard of living. However, diagnostic analysis reveals severe infrastructural disparity: while **Education (82.0)** acts as the strongest community pillar, **Transport (60.0)** presents an acute bottleneck that suppresses overall community progression. Comparative benchmarking demonstrates that **Area D** falls below the safe baseline threshold $(54.2 < 65.0)$, automatically triggering high-priority municipal alerts in the generated audit file.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

PO No.	Program Outcome	Project Specific Relevance & Justification
PO1	Engineering Knowledge	Applied mathematical modeling, statistics, algorithmic parsing, and SQL database operations to formulate civic indicator scoring engines.
PO2	Problem Analysis	Formulated multi-criteria evaluation matrices to diagnose municipal vulnerabilities and community infrastructure bottlenecks.
PO3	Design/Development	Engineered an automated modular system incorporating input sanitation, algorithmic max/min detection, and file generation.
PO4	Conduct Investigations	Conducted diagnostic experiments on multidimensional civic indicators to benchmark community living conditions.

PO5	Modern Tool Usage	Utilized Python 3 runtime, SQLite3 / MySQL databases, relational connectors, and automated file-handling libraries.
PO6	Engineer and Society	Directly addresses public infrastructure equity, citizen safety, and community health assessment for civic governance.
PO7	Environment & Sustainability	Integrates environmental metrics (air, sanitation, green space) to evaluate sustainability-linked municipal performance.

6.2 Sustainable Development Goals (SDGs) Mapping

SDG No.	Sustainable Goal	Direct Applicability to Project Workflow
SDG 3	Good Health and Well-being	Tracks community healthcare access and clinical response metrics to isolate underserved sectors.
SDG 4	Quality Education	Monitors school accessibility, literacy indices, and educational infrastructure reliability across communities
SDG 11	Sustainable Cities & Communities	Serves as the primary foundation for smart urban governance, resilient public transport, and civic indicator assessment
SDG 9	Industry, Innovation & Infrastructure	Fosters automated, data-driven modernization of municipal evaluation and essential public services.

7 Conclusion

The **Community Quality-of-Life Indicator Analyzer** successfully demonstrates the synergy of fundamental Python programming structures, algorithmic data validation,

and relational database management to resolve a critical municipal analytics challenge. By standardizing diverse indicators and generating automated diagnostic evaluations, the application eliminates manual bias and provides actionable insights for municipal authorities.

Future Enhancements: Future upgrades will involve integrating GIS spatial mapping (via GeoPandas / Folium) and deploying a RESTful web API for crowdsourced real-time telemetry.

Bibliography

- Ministry of Housing & Urban Affairs (MoHUA), Govt. of India, *Ease of Living Index Framework & Assessment Manual*.

- United Nations Development Programme (UNDP), *Human Development Report: Technical Notes on Composite Indices*.

- Python Software Foundation, *Python Standard Library Documentation (sqlite3, math, file I/O)*.